RNN + ATTENTION — A WORKED PICTURE

Where the numbers actually flow, timestep by timestep.

One pipeline (`“I love you”`), one zoom on the addition step, five sentences compared side by side, three epochs of the same sentence, when the weight matrices actually change, what gets lost without attention, why the summed vector carries no labels, and the gradient mechanics behind why “love” ends up mattering at all — all with the actual illustrative numbers, not just boxes and arrows.

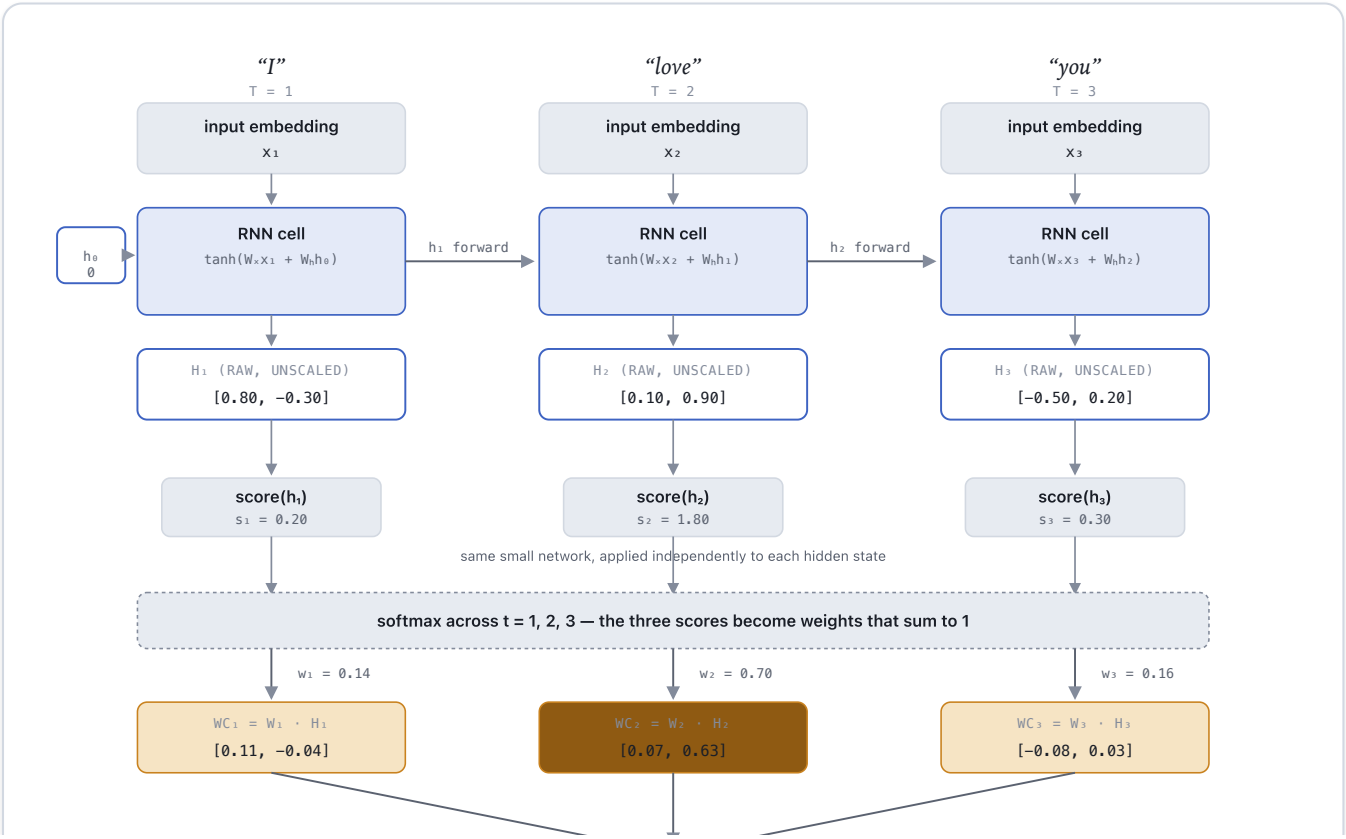
---

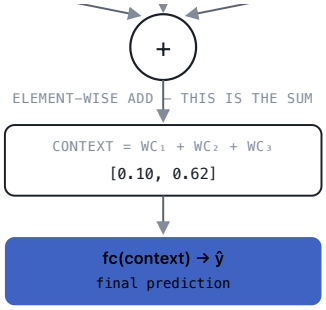
01 THE MECHANISM, UNROLLED

The RNN runs first. Attention looks back after.

The three hidden states `h₁, h₂, h₃` are produced strictly in order — `h₂` needs `h₁` to exist. Only once all three exist does the attention stage turn on: score each one, softmax across time, scale, then add.

hidden state (unscaled) attention weight — darker = more weight





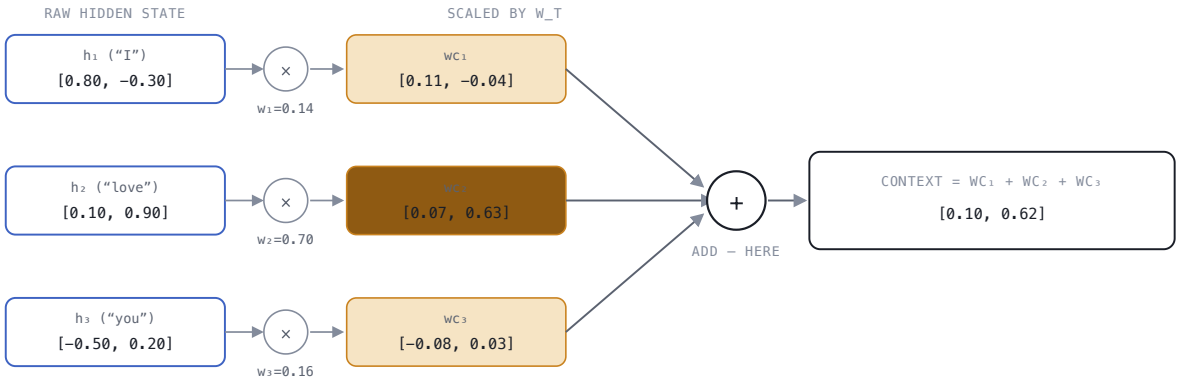
Notice the context vector's second component (**0.62**) is almost entirely wc_2's contribution (**0.63**) — "love" dominates the final answer because it got 70% of the attention weight, even though "I" and "you" also fed in their (much smaller) share.

---

02 THE ADDITION STEP, ZOOMED IN

h_t vs. w_t · h_t — and exactly where the "+" happens

Same numbers as above, laid out left to right so the transformation is explicit: **h_t** is the raw vector, untouched by attention. Multiplying it by the scalar **w_t** shrinks or stretches every entry — it's still the same shape, just rescaled. The "+" circle is the one and only place all three scaled copies get combined.



Left column and middle column are the **same size vector** — scaling by a scalar never changes dimensionality. The only step that combines information *across timesteps* is the single "+" node: everything before it is per-timestep, everything after it is sequence-level.

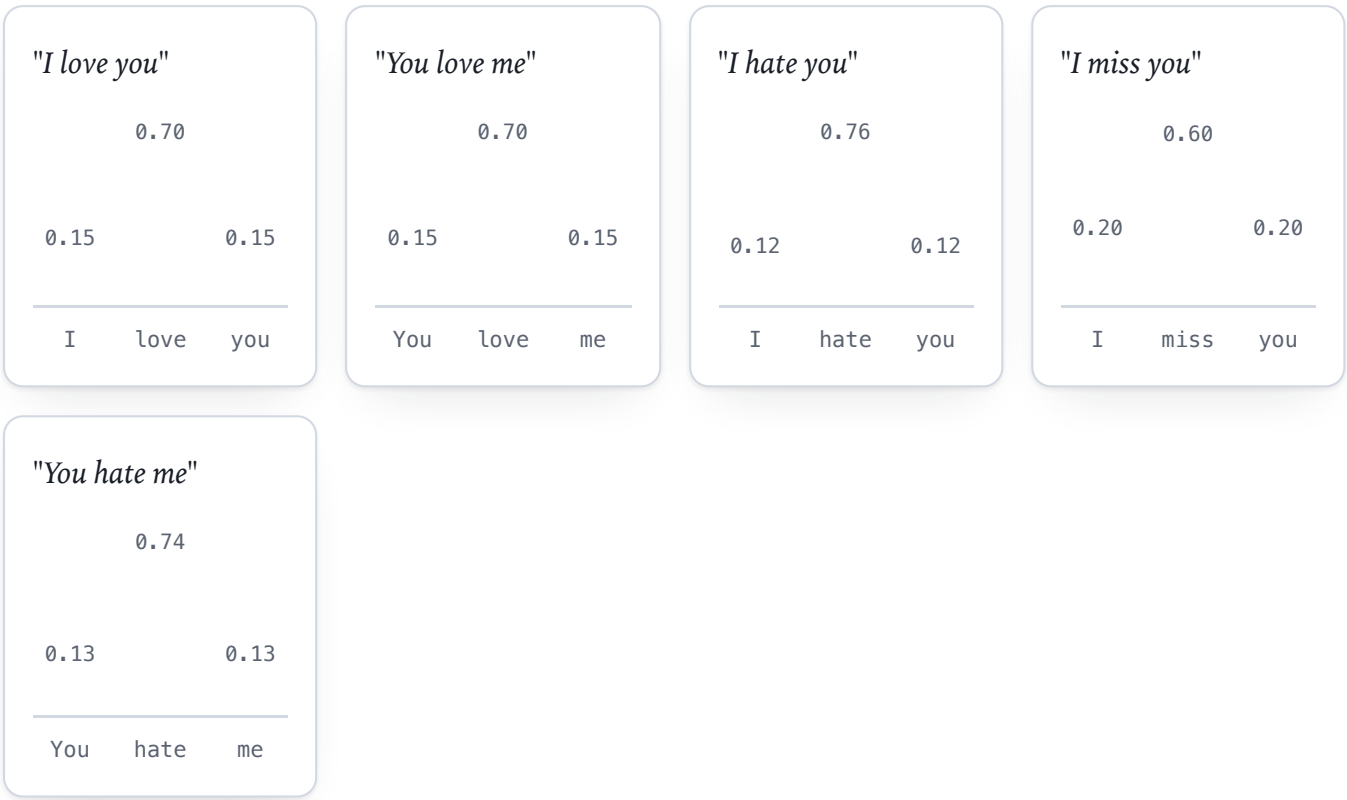
---

03 FIVE SENTENCES, SAME WEIGHTS

Same network, five inputs —

attention lands differently each time

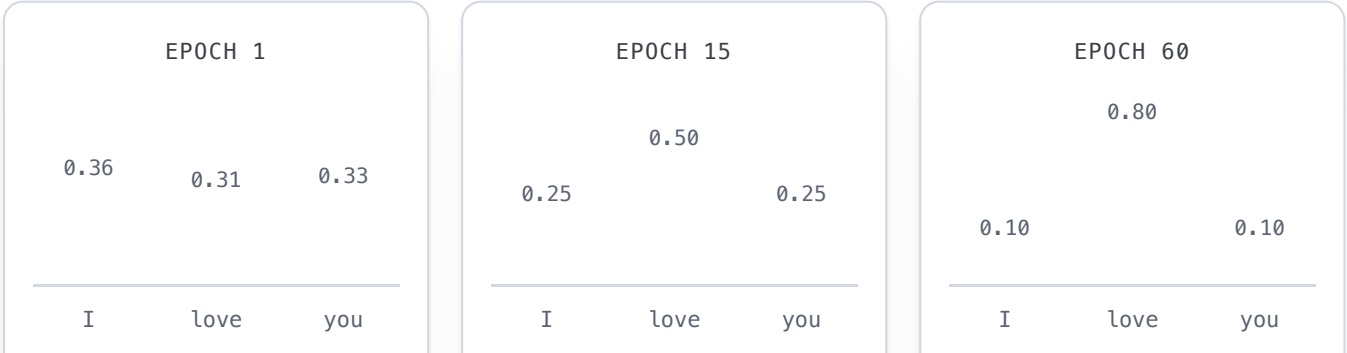
All five sentences share one set of weights (`W_x`, `W_h`, the score network). What changes per sentence is only the words — so the attention distribution it produces is different every time. Bars share one 0–0.8 scale so you can compare peaks directly.



04 WHAT CHANGES EVERY EPOCH

The loop never changes. The numbers inside it do.

Same sentence, same architecture, three snapshots of training on `"I love you"`. Early on, `attn_score` hasn't learned anything, so weights come out close to uniform. Backprop pushes it to sharpen toward whichever timestep the loss actually depends on.



attention hasn't learned where to look yet

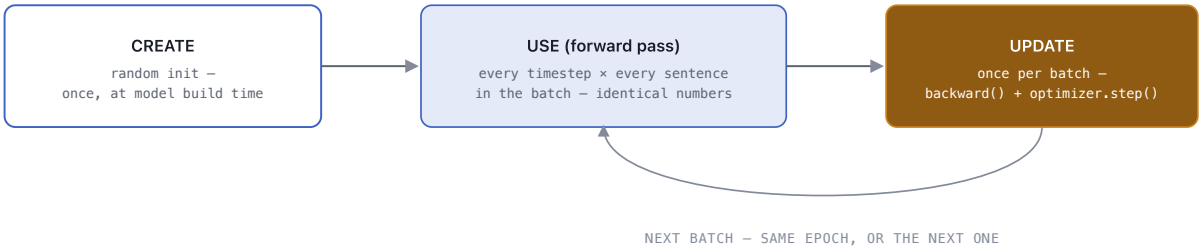
emerging

on "love"

05 WHEN THE WEIGHTS ACTUALLY CHANGE

W_x and W_h: created once, used everywhere, updated once per batch

Not "per sentence," not "per batch" — those are two different verbs. The same matrices are **used** (read, multiplied) identically for every sentence and every timestep inside a batch. They only **change value** once — after that batch's loss is computed and `optimizer.step()` runs. Then the cycle repeats with the nudged matrices.



This loop *is* `train_model()`: the middle box runs once for every sentence in `xb`, the right box runs exactly once per iteration of the batch loop — never mid-sentence, never per-timestep.

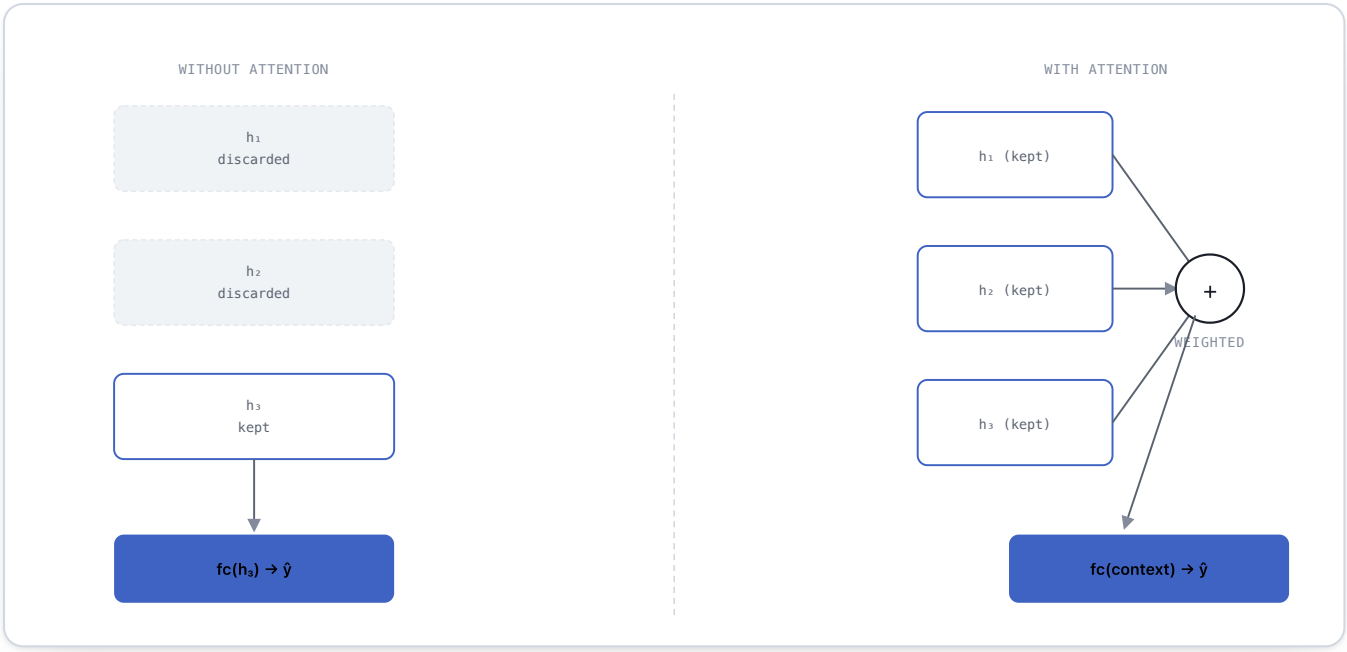
06 WHAT SURVIVES WITHOUT ATTENTION

Not concatenation — and not much survives without attention either

Attention doesn't stack `h₁, h₂, h₃` side by side (that would be concatenation). It adds them, scaled. But the more useful comparison is what the **baseline** RNN does instead: it doesn't add anything — it just keeps the last one.

discarded before the output layer even sees it — reaches the output layer

discarded before the output layer ever sees it reaches the output layer



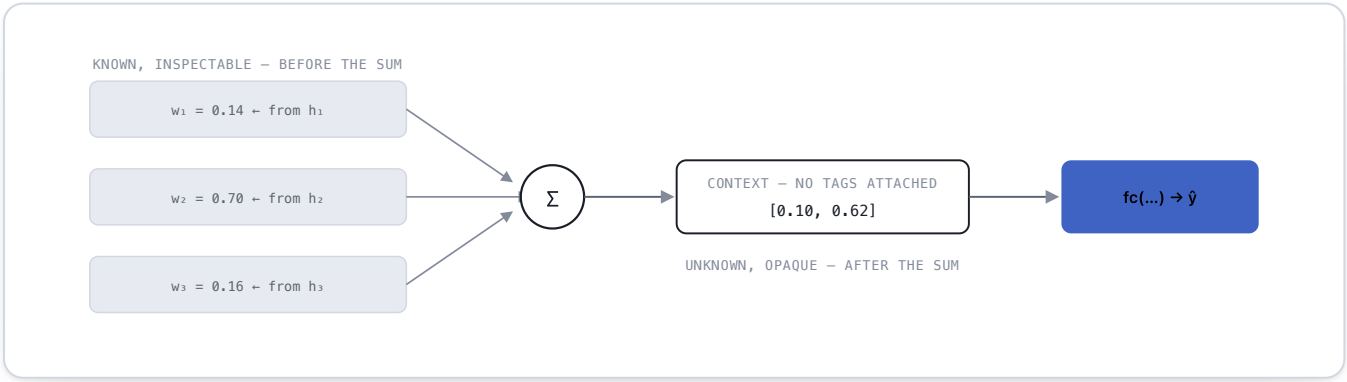
Both sides ultimately feed **one** vector into `fc`. The difference was never about how much survives to the very end — it's whether that one vector was forced through a fixed recurrence (left) or a learned, content-aware blend of everything (right).

---

07 AFTER THE SUM, NOTHING IS LABELED

The context vector carries no tags — only influence

Once `w₁, w₂, w₃` get summed into `context`, they're gone as separate objects. You cannot recover them from the result. What "importance" means past this point is only: a bigger `w_t` pushed `h_t`'s values harder into the final number — nothing is preserved as metadata.

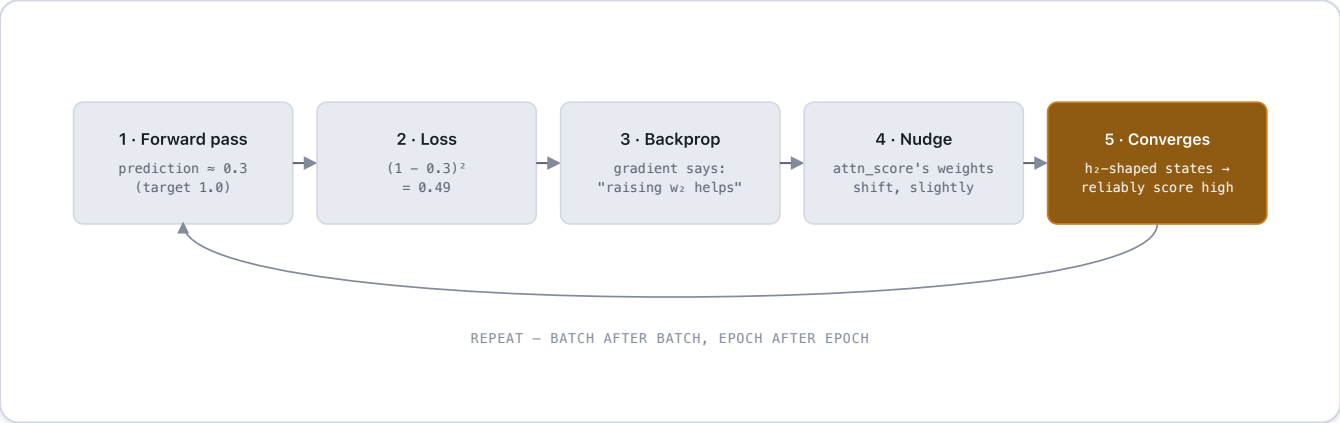


`fc` never receives `w₁, w₂, w₃` — only the two floats on the right. (Your notebook's `return_attention=True` saves the weights *on the side*, before the sum happens, purely for you to inspect — the model itself never gets that copy back.) The same principle is why Transformers add positional encoding *before* attention even runs: order has to be baked into the numbers up front, because nothing downstream can ask "which position was this."

08 WHY "LOVE" BECOMES POWERFUL

Nobody decided "love" matters — gradient descent discovered it was cheap

Every weight above — `attn_score`'s included — starts random. This loop, repeated thousands of times across training examples, is the entire mechanism by which "love" ends up with `w₂ = 0.70`.



Why h_2 specifically, mechanically: `context[1] = w₁·(−0.3) + w₂·(0.9) + w₃·(0.2)`. Since the `w`'s must sum to 1, shifting weight toward `w₂` — the only one multiplying a large *positive* number — is the cheapest lever gradient descent has for increasing `context[1]`, which `fc` had already learned to associate with a positive prediction. "Love" isn't special to the model; it's just the input that happened to make this particular lever effective, for this particular task.

All numbers on this page are illustrative, chosen to make the shapes clear — not values pulled from actually running the notebook. Sections 01–04 and 07–08 trace `AttentionRNN.forward()`: `self.rnn(x)` produces `h_t` for every t, `self.attn_score(out)` + `softmax` produces the weights, and `torch.bmm(weights.unsqueeze(1), out)` is the scale-and-add step. Section 05 traces `train_model()`'s batch loop. Section 06 contrasts it against `BaselineRNN`, which uses only `out[:, -1, :]`.